

Déploiement de l'API MadMen — Guide serveur

Configurer un serveur Ubuntu et y déployer l'API PHP `madmen-api-php`.

Couvre **3 approches** : Docker + Caddy (l'installation actuelle), Docker + Nginx, et **sans Docker** (Nginx + PHP-FPM natifs).

Mis à jour : 2026-06-22.

0. Ce qui est déjà en place (référence)

L'API tourne actuellement sur <https://api-madmen.ssmanager.uk> :

- **Serveur** : Hetzner Cloud, Ubuntu 24.04, IP `89.167.42.121`.
- **Reverse-proxy** : **Caddy** (en conteneur, projet `ssm-cloud`) → HTTPS automatique (Let's Encrypt).
- **Stack API** : conteneur PHP 8.4 (`madmen-api`) + conteneur MySQL 8.4 (`madmen-db`), Docker Compose.
- **DNS** : `ssmanager.uk` chez Cloudflare (DNS only), sous-domaine `api-madmen` → IP du serveur.

Les 3 méthodes ci-dessous mènent au même résultat ; choisis selon ton contexte (cf. §6).

1. Préparer le serveur (commun aux 3 méthodes)

Sur un VPS Ubuntu frais, en SSH :

```
# 1) Mises à jour
sudo apt update && sudo apt upgrade -y

# 2) Fuseau horaire (adapter)
sudo timedatectl set-timezone Africa/Brazzaville

# 3) Utilisateur non-root (si pas déjà fait)
sudo adduser deploy && sudo usermod -aG sudo deploy

# 4) Pare-feu : n'ouvrir que SSH + HTTP + HTTPS
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable

# 5) (recommandé) fail2ban contre le brute-force SSH
sudo apt install -y fail2ban
```

DNS : chez ton registrar / Cloudflare, crée un enregistrement **A** :

`api-madmen` → `<IP du serveur>` (chez Cloudflare : nuage **gris / DNS only** si tu utilises Caddy/certbot pour le TLS).

2. Méthode A — Docker + Caddy (recommandée, installation actuelle)

2.1 Installer Docker

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER # se reconnecter ensuite
```

2.2 Structure des fichiers (dans le dépôt `madmen-api-php`)

```

deploy/
  Dockerfile          # image PHP 8.4 qui sert public/index.php
  docker-compose.yml  # services api + db (+ adminer)
  bootstrap.sh        # déploie tout (clone, .env, build, migrations, route Caddy)
  .env                # secrets (NON committé) – monté dans le conteneur

```

`deploy/Dockerfile` (extrait clé) :

```

FROM php:8.4-cli-alpine
RUN apk add --no-cache linux-headers \
    && docker-php-ext-install pdo_mysql sockets
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install --no-dev --optimize-autoloader --no-scripts
COPY . .
RUN composer dump-autoload --no-dev --optimize && mkdir -p storage/uploads && chmod -R 0775 storage
EXPOSE 8000
ENV PHP_CLI_SERVER_WORKERS=4
CMD ["php", "-S", "0.0.0.0:8000", "-t", "public", "public/index.php"]

```

`deploy/docker-compose.yml` : un service `api` (build du Dockerfile, rejoint le réseau externe de Caddy) + `db` (MySQL, réseau privé). L'`api` monte le `.env` en lecture seule (`../.env:/app/.env:ro`).

2.3 Brancher au Caddy existant

Le Caddyfile (`/opt/ssm-cloud/deploy/Caddyfile.prod`) reçoit un bloc :

```

api-madmen.ssmanager.uk {
    reverse_proxy madmen-api:8000
}

```

Puis : `docker exec -w /etc/caddy ssm-caddy caddy reload --config /etc/caddy/Caddyfile`.

Caddy génère le **certificat HTTPS tout seul**. (Si pas de cert au bout d'1 min : `docker restart ssm-caddy` force une nouvelle tentative.)

2.4 Déployer / mettre à jour

```

cd /opt/madmen/madmen-api-php
git pull
docker compose -f deploy/docker-compose.yml --env-file .env up -d --build
docker compose -f deploy/docker-compose.yml --env-file .env exec api php database/migrate.php migrate

```

(Tout ceci est automatisé dans `deploy/bootstrap.sh` → `sudo bash bootstrap.sh`.)

3. Méthode B — Docker + Nginx (variante "standard" sans Caddy)

Architecture : **nginx** (proxy + TLS) → **php-fpm** (l'API) → **mysql**. Plus classique que le serveur PHP intégré.

`docker-compose.yml` :

```

name: madmen
services:
  app:
    # PHP-FPM (exécute l'API)
    build: { context: ., dockerfile: deploy/Dockerfile.fpm }
    volumes: [ "../.env:/app/.env:ro" ]
    networks: [ internal ]
    depends_on: { db: { condition: service_healthy } }

  web:
    # Nginx (sert public/ + proxy FastCGI vers app:9000)
    image: nginx:alpine
    ports: [ "80:80", "443:443" ]
    volumes:
      - ./deploy/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ./public:/app/public:ro
      - ./certs:/etc/nginx/certs:ro      # certificats (certbot ou autre)
    networks: [ internal ]
    depends_on: [ app ]

  db:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: madmen
      MYSQL_USER: madmen
      MYSQL_PASSWORD: ${DB_PASS}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
    volumes: [ "madmen_db:/var/lib/mysql" ]
    networks: [ internal ]
networks: { internal: {} }
volumes: { madmen_db: {} }

```

`deploy/Dockerfile.fpm` : identique au Dockerfile Caddy mais base `php:8.4-fpm-alpine` et **pas de CMD** `php -S` (php-fpm écoute sur 9000 par défaut).

`deploy/nginx.conf` :

```

server {
    listen 80;
    server_name api-madmen.ssmanager.uk;
    root /app/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string; # front controller
    }
    location ~ \.php$ {
        fastcgi_pass app:9000; # vers le conteneur php-fpm
        fastcgi_param SCRIPT_FILENAME /app/public$fastcgi_script_name;
        include fastcgi_params;
    }
}

```

HTTPS : soit un conteneur compagnon `certbot` (Let's Encrypt), soit un bloc `listen 443 ssl;` avec tes certificats montés dans `./certs`.

4. Méthode C — Sans Docker (Nginx + PHP-FPM natifs)

Tout est installé **directement** sur Ubuntu, sans conteneur.

4.1 Installer la pile

```

# Dépôt PHP récent
sudo add-apt-repository ppa:ondrej/php -y && sudo apt update
sudo apt install -y nginx php8.4-fpm php8.4-mysql php8.4-mbstring \
    php8.4-xml php8.4-curl php8.4-sockets \
    mysql-server git unzip

# Composer
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer

```

4.2 Récupérer le code

```
sudo mkdir -p /var/www && cd /var/www
sudo git clone -b feat/controle-activite-fondations \
https://github.com/sapieproductionsllc-afk/madmen-api-php.git
cd madmen-api-php
sudo composer install --no-dev --optimize-autoloader
sudo chown -R www-data:www-data /var/www/madmen-api-php/storage
```

4.3 Base de données

```
sudo mysql -e "CREATE DATABASE madmen CHARACTER SET utf8mb4;"
sudo mysql -e "CREATE USER 'madmen'@'localhost' IDENTIFIED BY 'MOT_DE_PASSE';"
sudo mysql -e "GRANT ALL ON madmen.* TO 'madmen'@'localhost'; FLUSH PRIVILEGES;"
```

Puis créer `.env` (à partir de `deploy/.env.server.example`) avec `DB_HOST=127.0.0.1`, `DB_USER=madmen`, etc., et lancer :

```
php database/migrate.php migrate
```

4.4 Vhost Nginx

`/etc/nginx/sites-available/api-madmen :`

```
server {
    listen 80;
    server_name api-madmen.ssmanager.uk;
    root /var/www/madmen-api-php/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.4-fpm.sock;
    }
    location ~ /\.(!well-known) { deny all; } # protège .env, .git, etc.
}
```

```
sudo ln -s /etc/nginx/sites-available/api-madmen /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

4.5 HTTPS (Let's Encrypt)

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d api-madmen.ssmanager.uk
# Renouvellement auto déjà configuré (timer systemd).
```

4.6 Mise à jour

```
cd /var/www/madmen-api-php
sudo git pull
sudo composer install --no-dev --optimize-autoloader
php database/migrate.php migrate
sudo systemctl reload php8.4-fpm
```

5. Configuration `.env` de production (les 3 méthodes)

```

APP_ENV=production
APP_DEBUG=false
APP_KEY=&lt;64 hex&gt;      # openssl rand -hex 32 (signe aussi les JWT)
API_KEY=&lt;48 hex&gt;      # openssl rand -hex 24
AUTH_ENABLED=true
BRUTE_FORCE_ENABLED=true
BIO_SIMULATION=false
CORS_ORIGIN=https://dashboard.ton-domaine # JAMAIS '*' en prod (refusé au boot)
DB_HOST=db                # 'db' en Docker, '127.0.0.1' en natif
DB_PORT=3306
DB_NAME=madmen
DB_USER=madmen
DB_PASS=&lt;mot de passe fort&gt;

```

■ ■ L'API **refuse de démarrer** en production si la config est dangereuse (**AUTH_ENABLED=false**, secrets par défaut, ou **CORS_ORIGIN=***).

6. Quelle méthode choisir ?

Méthode	Avantages	Inconvénients	Pour qui
A. Docker + Caddy	HTTPS auto, isolé, reproductible, 1 commande	Le serveur PHP intégré est simple (suffisant petite charge)	Recommandé ici (déjà en place)
B. Docker + Nginx	Standard prod (nginx+fpm), perfs, isolé	Plus de fichiers de config, HTTPS à gérer	Charge plus élevée, équipe habituée à nginx
C. Natif (sans Docker)	Pas de couche Docker, contrôle total	Installe tout à la main, dépendances système, moins reproductible	Petit VPS dédié, pas de Docker

7. Pièges rencontrés (retour d'expérience)

- **Extension PHP sockets** sous Alpine → ajouter `apk add linux-headers` avant `docker-php-ext-install`.
- **Caddy reste en backoff** après un échec de certificat → `docker restart ssm-caddy` force une nouvelle tentative (réussit via TLS-ALPN).
- **CORS_ORIGIN=*** est refusé en production → mettre une liste d'origines concrètes (l'API gère une liste séparée par virgules).
- **Hash bcrypt corrompu** quand on colle un hash dans un terminal/SQL (retour à la ligne inséré) → générer le hash côté serveur (PHP), jamais le coller.
- **Commandes longues** collées dans un terminal SSH se coupent → mettre la logique dans des **scripts** et ne coller que des commandes courtes.
- **Disque plein** bloque le build Docker → `docker system prune -f` pour faire de la place.

8. Sauvegardes (à ne pas oublier)

```

# Dump de la base (Docker)
docker compose -f deploy/docker-compose.yml exec -T db \
  mysqldump -uroot -p"$DB_ROOT_PASSWORD" madmen &gt; backup_$(date +%F).sql

# + sauvegarder le dossier storage/uploads (pièces jointes messagerie)

```

À planifier via **cron** (quotidien) et copier hors-serveur.